

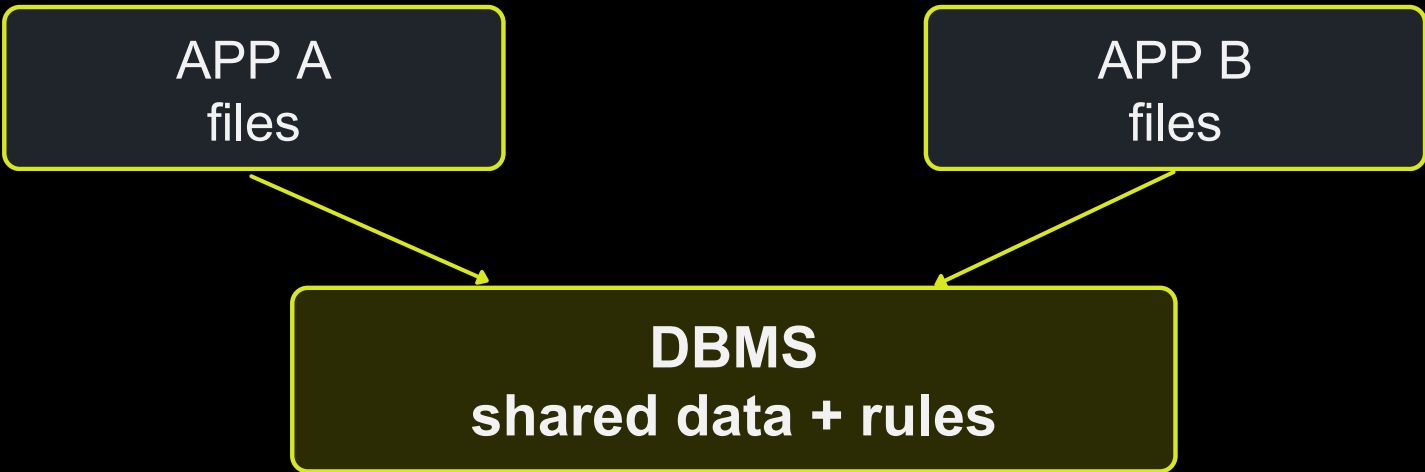
1.

What is the difference between a DBMS and a traditional file system?

- File system stores raw files; DBMS manages structured data with metadata.
- DBMS provides querying, security, concurrency control and recovery.
- Avoids uncontrolled redundancy and inconsistent copies of the same data.
- Best interview angle: DBMS = data + rules + access control + reliability.

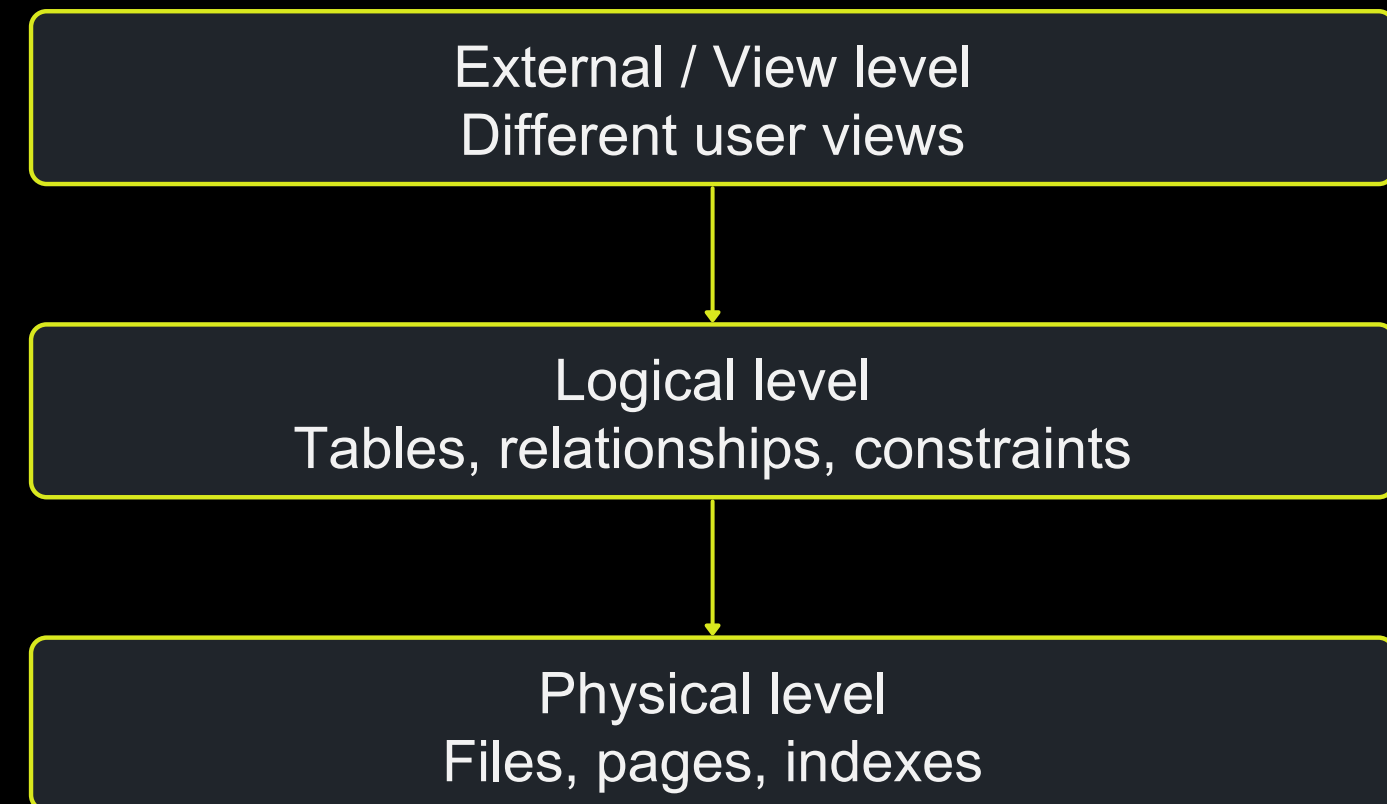
FILE SYSTEM VS DBMS

Basis	File system	DBMS
Access	Manual programs	Query language
Integrity	App-level	Constraints
Concurrency	Limited	Built-in control
Recovery	Manual backups	Logs + recovery



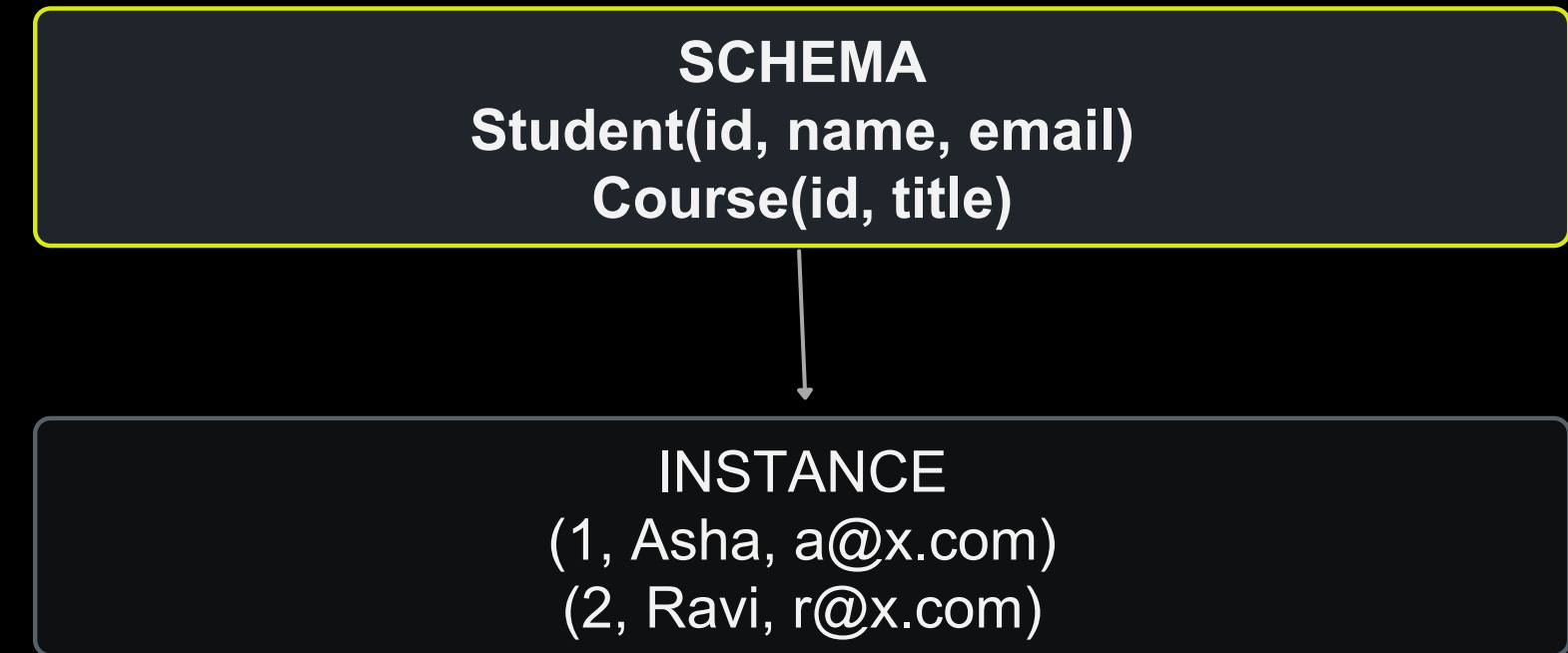
2. What is data independence, and why is it important in database systems?

- **Data independence means changing one level without breaking other levels.**
- **Physical independence: storage/index changes do not affect logical schema.**
- **Logical independence: logical schema changes minimally affect user views.**
- **It reduces application rewrites and makes DB systems easier to evolve.**



3. How is a database schema different from a database instance?

- Schema is the database design/structure: tables, columns and constraints.
- Instance is the actual data stored at a particular moment.
- Schema changes rarely; instance changes whenever rows are inserted/updated.
- Analogy: schema is blueprint, instance is the current building state.



DIFFERENCE

Point	Schema	Instance
Meaning	Structure	Current data
Changes	Rarely	Frequently
Example	Columns	Rows

4. What is the difference between a primary key and a candidate key?

- Candidate keys are all minimal attributes that can uniquely identify a row.
- Primary key is the candidate key chosen as the main row identifier.
- A table can have many candidate keys but only one primary key.
- Primary key should be stable, unique, non-null and preferably small.

STUDENT TABLE

roll_no	email	phone
101	a@x.com	9991
102	b@x.com	9992
103	c@x.com	9993

Candidate keys
{roll_no}, {email}, {phone}

Chosen primary key
roll_no

ALL PKs ARE CANDIDATE KEYS

5. What is referential integrity, and how does a DBMS enforce it?

- Referential integrity keeps child-table foreign keys pointing to valid parent rows.
- A child row cannot reference a parent record that does not exist.
- DBMS enforces it through foreign key constraints.
- On parent delete/update, actions may be **RESTRICT**, **CASCADE**, **SET NULL** or **SET DEFAULT**.

STUDENTS
id PK
name

ENROLLMENTS
student_id FK
course_id FK

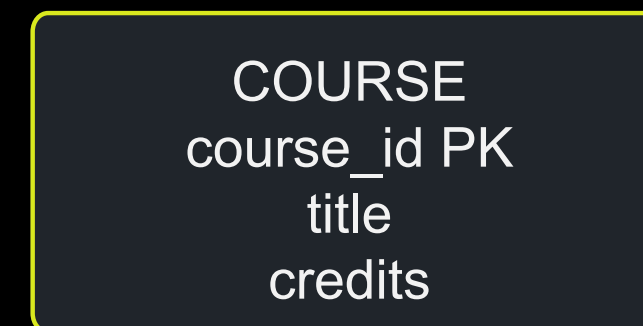
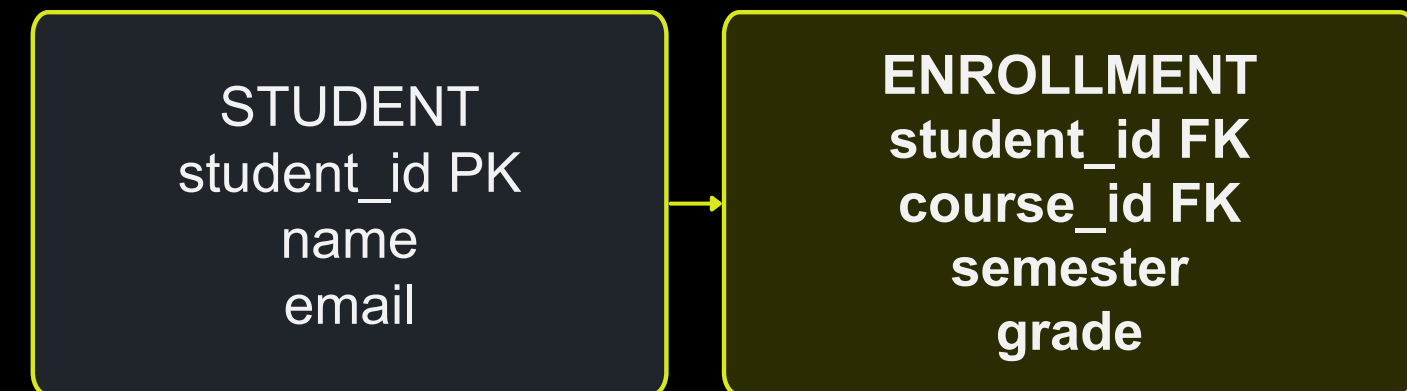
PARENT ACTION

Action	Meaning
RESTRICT	Block delete
CASCADE	Delete child rows
SET NULL	Keep child, clear FK

**Without valid parent
foreign key insert fails**

6. You are designing a college database. How would you model students, courses, and enrollments?

- Identify entities first: Student and Course are independent entities.
- Enrollment represents the relationship between a student and a course.
- Use Enrollment(student_id, course_id) as a bridge table with foreign keys.
- Keep relationship attributes like grade, semester and enrollment_date there.



M:N becomes two 1:N relationships

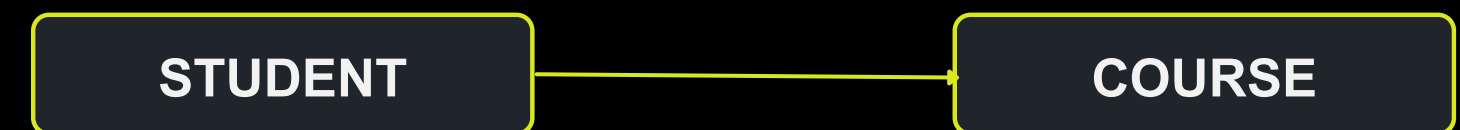
BRIDGE KEEPS RELATIONSHIP DATA

Attribute	Placed in
grade	Enrollment
semester	Enrollment
credits	Course

7. Why is a many-to-many relationship usually resolved using an associative entity?

- Relational tables cannot directly store a clean M:N relationship without repetition.
- An associative entity converts M:N into two manageable 1:N relationships.
- It prevents repeated groups and supports extra relationship attributes.
- It also makes constraints, indexing and querying cleaner.

BEFORE: DIRECT M:N



AFTER: ASSOCIATIVE ENTITY



WHY IT HELPS

Benefit	Impact
No repetition	Cleaner rows
FKs enforced	Better integrity
Attrs stored	grade, semester

8. What are insertion, update, and deletion anomalies?

- Anomalies happen when a poor table design mixes multiple facts in one table.
- **Insertion anomaly:** cannot add one fact without unrelated data.
- **Update anomaly:** same fact appears in many rows and must be changed everywhere.
- **Deletion anomaly:** deleting one row accidentally removes another important fact.

BAD TABLE: STUDENTCOURSE

Student	Course	Teacher
Asha	DBMS	Rao
Ravi	DBMS	Rao
Neha	CN	Mehta

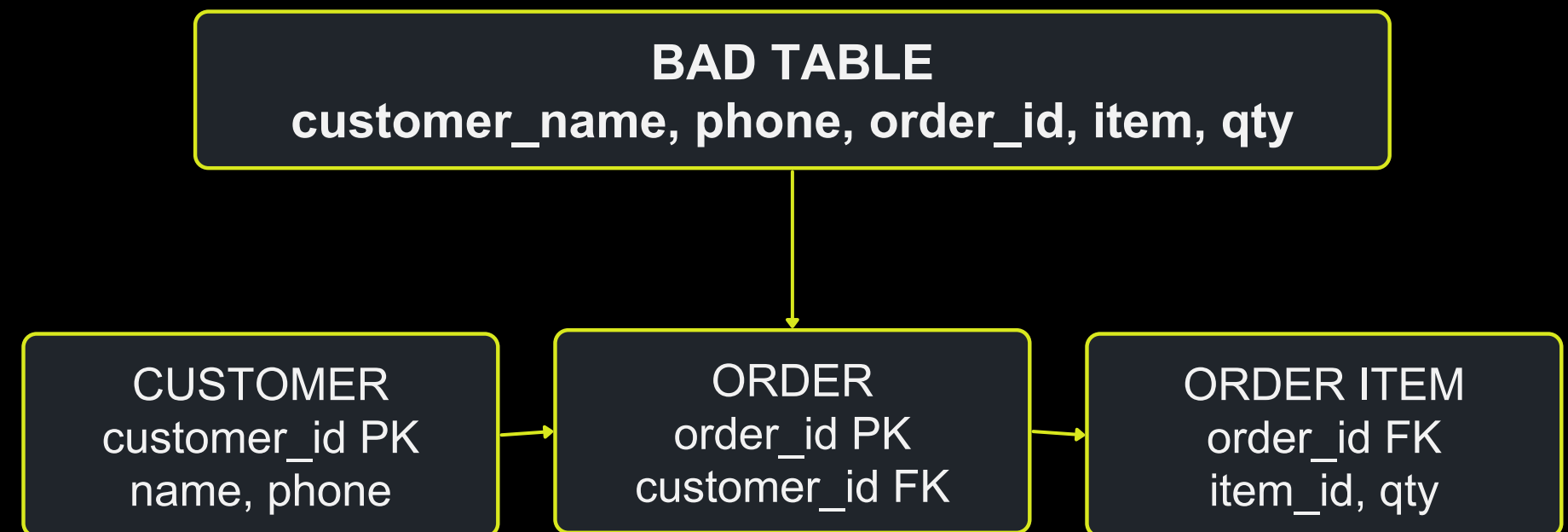
Update anomaly
Teacher name repeated

Deletion anomaly
Remove last CN student → lose CN teacher

Insertion anomaly
Cannot add new course without student

9. A table stores customer details and repeated order information. How would you decide whether normalization is required?

- Look for repeated groups, duplicate customer data and mixed facts in one row.
- Check update/delete/insert anomalies and functional dependencies.
- Split stable entities: Customer, Order and OrderItem.
- Normalize when correctness and maintainability matter more than one-table convenience.



DECISION CHECKS

Check	Meaning
Repeating values	Normalize
Many updates	Normalize
Pure reporting	Maybe denormalize

10. What is the difference between Third Normal Form and BCNF?

- 3NF removes transitive dependency but allows a special case when RHS is prime.
- BCNF is stricter: for every functional dependency $X \rightarrow Y$, X must be a super key.
- Every BCNF table is in 3NF, but every 3NF table may not be in BCNF.
- BCNF reduces redundancy further but can sacrifice dependency preservation.

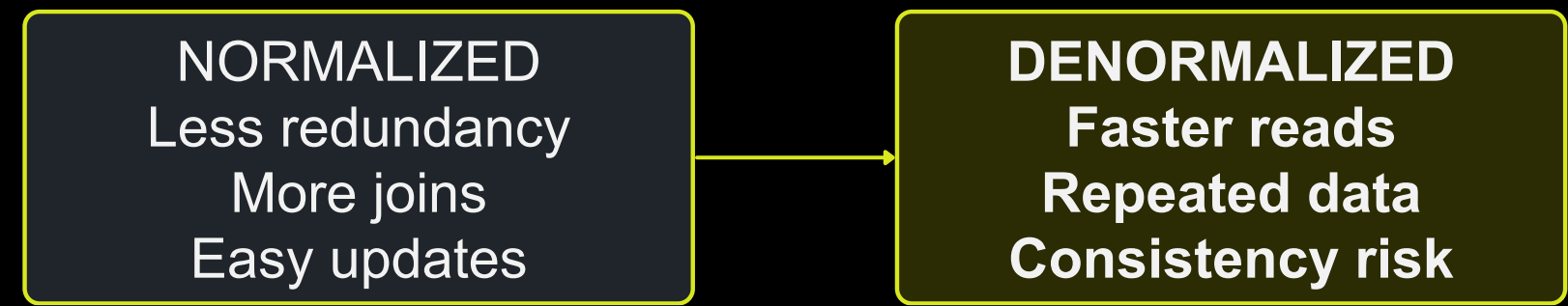
3NF VS BCNF

Basis	3NF	BCNF
Rule	X super key OR Y prime	X must be super key
Strength	Less strict	More strict
Always?	BCNF $\Rightarrow$ 3NF	3NF $\nRightarrow$ BCNF
Trade-off	Dependency friendly	Less redundancy

Interview line
BCNF = stronger version of 3NF where every determinant is a candidate/super key.

11. When can denormalization be a better design choice than normalization?

- Denormalization adds controlled redundancy to improve read performance.
- Useful for dashboards, reports, materialized summaries and read-heavy systems.
- It reduces joins but increases storage and consistency-management cost.
- Never denormalize blindly; do it for measured access patterns.



USE IT WHEN

Situation	Reason
Dashboard	Precomputed summary
Heavy reads	Avoid repeated joins
Low writes	Less update pain

Trade correctness cost for read speed carefully

12. What do the ACID properties guarantee in a database transaction?

Atomicity: all operations complete or none do.

Consistency: transaction moves database from one valid state to another.

Isolation: concurrent transactions should not corrupt each other.

Durability: committed changes survive crashes.

Atomicity
All or nothing

Consistency
Valid rules

Isolation
Safe concurrency

Durability
Survives crash

Bank transfer
Debit A + Credit B
commit together

13. Two users try to update the same account balance at the same time. What concurrency problem can occur?

- This can cause the lost update problem.
- Both transactions read the old balance, then overwrite each other's result.
- Final balance becomes incorrect because one update is effectively lost.
- DBMS prevents it using isolation, locks, MVCC checks or atomic update logic.

LOST UPDATE SCHEDULE

Step	T1	T2
1	Read balance = 100	
2		Read balance = 100
3	Write 90	
4		Write 80
Final	Expected 70	Actual 80

Problem
T2 overwrites T1 update

Concurrency control is needed

14. What is the difference between a serial schedule and a serializable schedule?

- Serial schedule executes transactions one after another with no interleaving.
- Serializable schedule may interleave operations but final result equals some serial order.
- Serializable schedules improve concurrency while preserving correctness.
- Interview key: not every concurrent schedule is serializable.

SERIAL
T1 completes
then T2 starts

SERIALIZABLE
T1 and T2 interleave
but result is safe

EXECUTION IDEA

Type	Interleaving	Correctness
Serial	No	Safe
Serializable	Allowed	Same as serial
Non-serializable	Allowed	May be wrong

15. What is a dirty read, and which isolation level allows it?

- Dirty read means reading data written by another transaction before it commits.
- If the writer later rolls back, the reader has used a value that never existed permanently.
- READ UNCOMMITTED can allow dirty reads.
- READ COMMITTED prevents dirty reads in normal SQL isolation behavior.

DIRTY READ TIMELINE

Step	T1	T2
1	Update balance = 500	
2		Read balance = 500
3	ROLLBACK	
Issue		T2 read uncommitted data

Allowed by
READ UNCOMMITTED

Prevented by
READ COMMITTED or higher

16. A transaction reads the same row twice but gets different values. Which anomaly has occurred?

- This is a non-repeatable read.
- T1 reads a row, T2 updates and commits it, then T1 reads again and gets a new value.
- It is different from phantom read, where new matching rows appear/disappear.
- Repeatable Read or stronger isolation is generally used to prevent it.

NON-REPEATABLE READ

Step	T1	T2
1	Read salary = 50k	
2		Update salary = 60k
3		COMMIT
4	Read salary = 60k	

DO NOT CONFUSE WITH

Anomaly	What changes
Dirty read	Uncommitted value
Non-repeatable	Same row value
Phantom	Number of rows

17. What is a deadlock in DBMS, and how can the database detect it?

- Deadlock happens when transactions wait for each other in a cycle.
- Example: T1 holds A and waits for B; T2 holds B and waits for A.
- DBMS detects it using a wait-for graph; a cycle means deadlock.
- Recovery usually aborts one victim transaction and releases its locks.



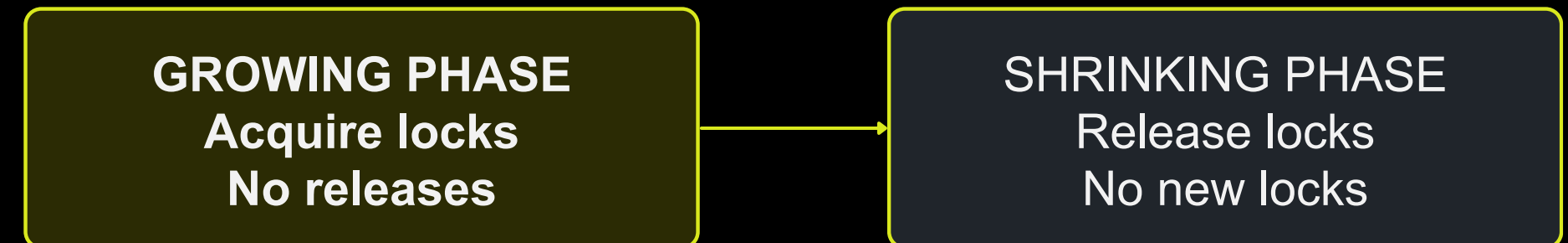
WAIT-FOR GRAPH CYCLE

Detect cycle
→ choose victim
→ rollback victim
→ release locks

18. How does Two-Phase Locking help maintain serializability?

- 2PL controls when a transaction can acquire and release locks.
- Growing phase: transaction can acquire locks but cannot release any.
- Shrinking phase: transaction can release locks but cannot acquire new ones.
- This discipline guarantees conflict serializability, though it may still deadlock.

TWO PHASES



VARIANTS

Variant	Idea
Basic 2PL	Two phases only
Strict 2PL	Hold write locks till commit
Conservative	Get locks before start

19.

How does MVCC improve concurrency compared with strict lock-based execution?

- MVCC keeps multiple versions of rows instead of forcing every read to wait.
- Readers can read a consistent snapshot while writers create new versions.
- This reduces read-write blocking and improves throughput for read-heavy systems.
- Old versions are eventually cleaned after no transaction needs them.

LOCKING VS MVCC

Point	Strict locks	MVCC
Read/write	Often blocks	Snapshot read
Writers	Lock rows	Create versions
Best for	Simple control	High concurrency
Cost	Waiting	Version cleanup

Row version chain
V1 → V2 → V3

READER SEES SNAPSHOT, WRITER CONTINUES

20. A table has millions of records and searches are becoming slow. What DBMS feature would you consider first?

- First consider an index on columns used in WHERE, JOIN, ORDER BY or GROUP BY.
- The index lets DBMS avoid scanning every row for selective searches.
- Choose index based on query pattern, selectivity and sort requirements.
- Still verify using the execution plan; not every slow query is fixed by an index.

Full table scan
checks millions of rows

Index lookup
jumps to matching rows

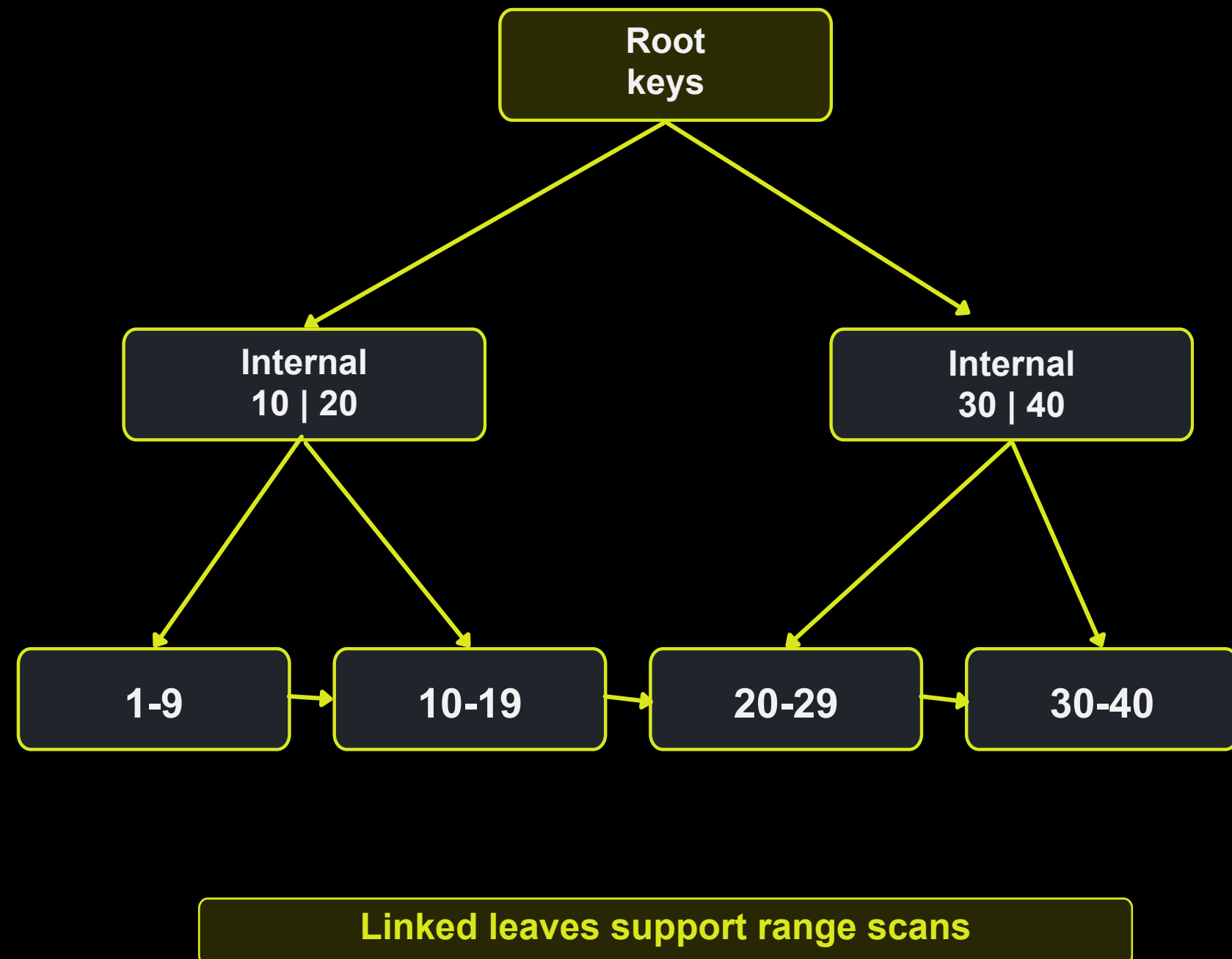
INDEX DECISION

Column use	Index value
WHERE filter	High
JOIN key	High
ORDER BY	Useful
Low selectivity	Maybe low

Always confirm with EXPLAIN

21. Why are B+ trees commonly used for database indexes?

- B+ trees are balanced, so lookup time stays predictable as data grows.
- Internal nodes guide search; actual records/row pointers stay at leaf level.
- Leaf nodes are linked, making range queries and ordered scans efficient.
- They work well for equality search, range search, sorting and disk-page access.



22. What is the difference between clustered and non-clustered indexing?

- **Clustered index** defines the physical/logical order of table data.
- A table usually has only one clustered organization because rows have one order.
- **Non-clustered index** is a separate structure that points to data rows.
- Clustered helps range reads; non-clustered supports multiple alternate lookups.

CLUSTERED VS NON-CLUSTERED

Basis	Clustered	Non-clustered
Data order	Rows ordered by index	Separate index
Count	Usually one	Can be many
Leaf level	Actual data	Row pointer/key
Best for	Range scans	Lookups

Clustered: index order = data order

Non-clustered: index points to row

23. Why can adding too many indexes reduce database performance?

- Every INSERT, UPDATE and DELETE may need to update multiple index structures.
- Indexes consume extra disk and memory.
- Too many indexes increase maintenance cost and can slow write-heavy workloads.
- Good answer: add indexes for proven access patterns, not for every column.

READS
faster filtering
less scanning

WRITES
more index updates
slower commits

INDEX COST

Cost	Why it matters
Storage	Extra pages
Insert	Add index entry
Update	Possibly move entries
Delete	Remove entries

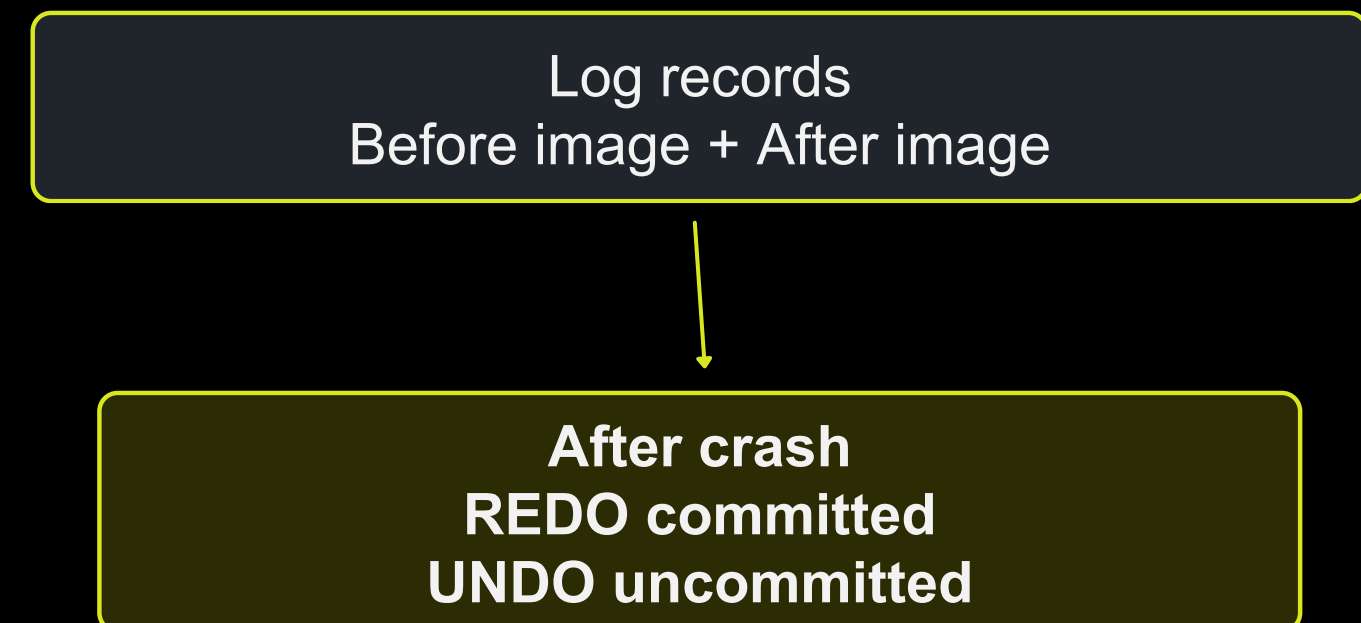
Index only what queries actually need

24. A server crashes while a transaction is partially completed. How does log-based recovery restore consistency?

- DBMS uses transaction logs to know what happened before the crash.
- Committed changes are redone if they had not reached the data page.
- Uncommitted changes are undone so partial work does not remain.
- Recovery ensures atomicity and durability after failure.

CRASH RECOVERY DECISION

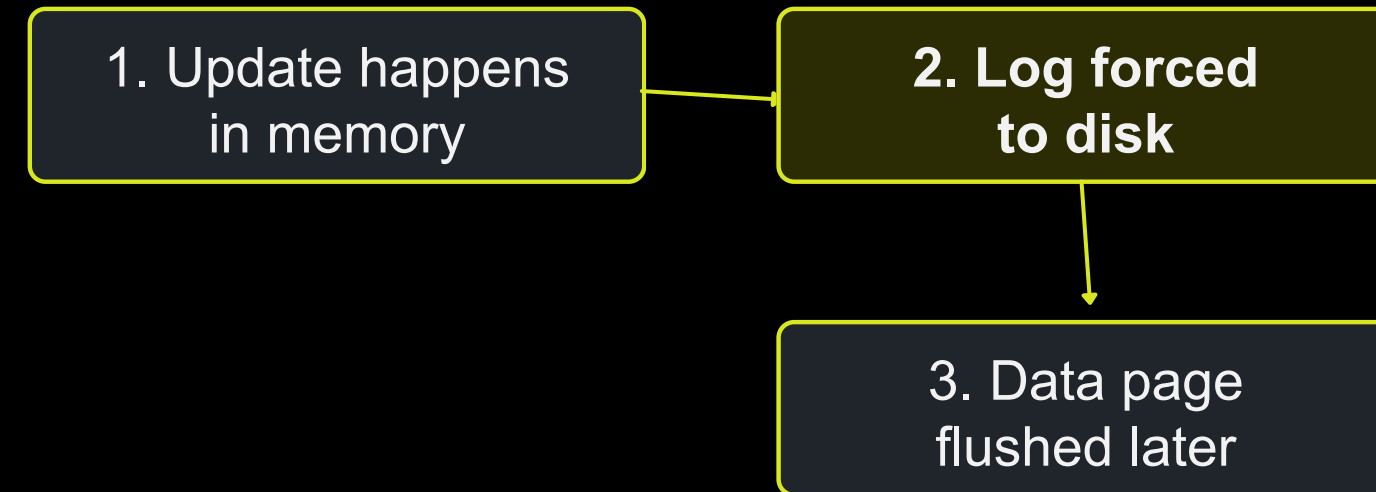
Txn state	Recovery action
Committed	REDO if needed
Uncommitted	UNDO
Checkpointed	Less log scan
Partial write	Use log record



25. What is Write-Ahead Logging, and why must the log be written before the data page?

- **Write-Ahead Logging** means log records are safely written before data pages are flushed.
- If the system crashes, the log becomes the source for redo and undo recovery.
- Without WAL, a data page may contain changes that cannot be explained or reversed.
- Rule: log first, data later; commit only after required log records are durable.

WAL ORDER



WHY LOG FIRST?

Guarantee	Meaning
Atomicity	Undo partial work
Durability	Redo committed work
Consistency	Recover known state